FINAL-YEAR PROJECT | CIE-3

TEAM RSST

Rohit Jaysheel Diggi	1RVU23CSE384
Saran Chekka	1RVU23CSE411
Sushrith Kandagatla	1RVU23CSE489
Tamish Sridatta Billa	1RVU23CSE498

ShieldWall

An eBPF/XDP engine that stops DDoS attacks on small servers in seconds and explains itself in plain English with gpt-4o-mini.

C | Python | eBPF/XDP

Detect, Mitigate, Explain

Small servers drown in fake traffic

37%

of all DDoS attacks hit the gaming industry, the most attacked sector

$38/hr

is all a DDoS-for-hire service costs. Attacking is cheap, defending is not

94%

of web DDoS attacks stay under 100k req/s, small enough to slip past basic detection

Our focus: small operators (game servers, small VPS owners, campus networks) who can't afford Cloudflare-scale protection and have nothing automatic today.

Sources: Akamai State of the Internet, SentinelOne, StationX DDoS Statistics 2026, Cloudflare

What the system has to do

1. **Detect fast.** Spot a flood within seconds from live traffic counts.
2. **Block automatically.** Drop attack traffic in the kernel with no human in the loop.
3. **Protect real users.** Allowlist, rule cap and expiring rules so players keep playing.
4. **Explain itself.** gpt-4o-mini writes plain-English summaries but never makes blocking decisions.
5. **Run on small hardware.** One commodity server, tested in an isolated lab.

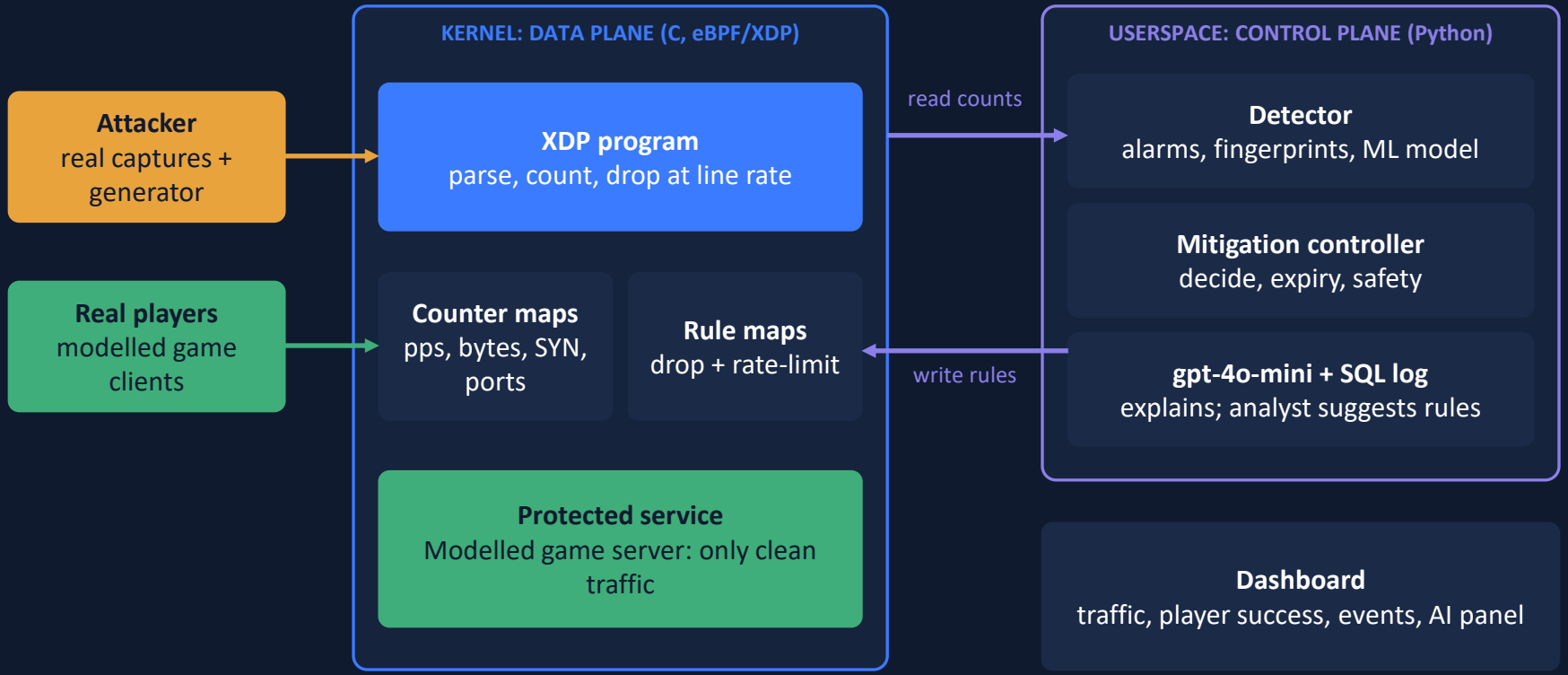
Known limit

Floods bigger than the internet link need upstream help. FlowSpec escalation is shown in our simulator, not on a real ISP.

Training: ML only

Alarms learn normal traffic live. Only the ML tier is trained, for unnamed attacks.

Design: the kernel drops, userspace decides



Implementation: detect, mitigate, explain

Detect

Vote, then name it

- EWMA baseline with z-score
- CUSUM catches slow ramps
- Entropy of source spread
- Count-Min, HyperLogLog and Bloom sketches track sources in KBs
- Named by fingerprints, then a trained ML model

Mitigate

Block, then undo

- Signature drop per vector
- Rate limit unknown sources
- Heavy-hitter + fragment rules
- Every rule expires in ~60 s
- Allowlist, rule cap, dry-run

Explain

Off the hot path

- gpt-4o-mini writes the incident summary
- Chat over the SQL event log
- AI analyst suggests rules; controller tests them
- Never makes a blocking decision

Stack: C + eBPF/XDP (kernel), Python + bpftool (control), scikit-learn (ML tier), SQLite, OpenAI gpt-4o-mini, scapy kernel test, live web dashboard, pytest + GitHub Actions

Evaluation and results

Real attack data

1. Replayed attacks: 6/6 caught
2. Real kernel: 98.5-99.1% dropped
3. Held-out: 5/8 caught
4. SYN-ACK miss: named by ML

LLM check (gpt-4o-mini)

100% numbers grounded (66/66)
100% faithful on unseen incidents
1.5 s reply, **$0.0001** each

Simulated lab	Without	With ShieldWall
Time to detect (s)	No detection	**2 s**
Time to mitigate (s)	Never	**3-4 s**
Real traffic served in attack	3-50%	**95.7-98.6%**
False alarms (93 non-attacks)	-	**0**
Detection F1 (200 trials)	-	**F1 0.99**

Real Linux kernel: real DNS and multi-vector captures through the XDP program: 98.5% and 99.1% of packets dropped; the SYN-ACK capture, named by ML: 98.4% of its SYN-ACKs. 0 player packets. Table results come from our simulator.

Code and summary

GitHub repository

github.com/SushrithKbtech/shieldwall

XDP program, detection engine, ML model, dashboard, tests and full results.

Real problem: Gaming is the top DDoS target and small operators are priced out of protection.

Real engineering: Kernel eBPF/XDP, statistical alarms, a trained ML tier and a grounded gpt-4o-mini explainer.

Real proof: real captures blocked by the real XDP kernel (98.5-99.1% dropped); the held-out SYN-ACK miss is now caught by ML.

A smart bouncer for small servers: it blocks attacks by itself and explains what it did.